UNIWERSYTET GDAŃSKI · WMFiI · TESTOWANIE AUTOMATYCZNE

Monitor basenów

Lab 1 Uruchomienie projektu i pierwsze testy

1 Cel zadania

W kompleksie basenowym czujniki mierzą temperaturę wody. Skrypt w Pythonie pobiera odczyt z serwera i decyduje, co zrobić: nic (OK), podgrzać, schłodzić albo wywołać alarm. Twoje zadanie to uruchomić projekt i napisać do niego automatyczne testy w pytest. Zajmie to około 30 minut.

2 Co zawiera projekt

Repozytorium: github.com/konrad-ug/basen-monitorig. Są w nim tylko cztery rzeczy, które Cię interesują:

- `monitor.py` – program (logika decyzji o akcji dla basenów),
- `tests/test_basen.py` – testy; tu będziesz pracować,
- `requirements.txt` – lista paczek (pytest, requests),
- `README.md` i `pytest.ini` – opis i ustawienia pytest (nie musisz ich zmieniać).

3 Uruchomienie

Krok 1: pobierz kod

Wybierz jedną opcję: **git clone**, pobranie **ZIP** (Code → Download ZIP, potem rozpakuj) albo **Codespaces** (Code → Codespaces, praca w przeglądarce).

```
git clone https://github.com/konrad-ug/basen-monitorig.git
cd basen-monitorig
```

Kolejne polecenia wykonuj w folderze projektu (tam, gdzie leży `pytest.ini`). Potrzebujesz Pythona 3.9 lub nowszego.

Krok 2: zainstaluj paczki

```
pip install -r requirements.txt
```

Krok 3: uruchom testy

```
pytest                # uruchamia wszystkie testy
pytest -v             # to samo, z listą każdego testu
pytest -k jacuzzi     # tylko testy z "jacuzzi" w nazwie
python monitor.py     # wypisuje decyzję dla każdego basenu
```

Zobaczysz **5 zielonych testów** (komunikat `5 passed`). Jeśli tak jest, możesz zaczynać zadanie.

> **Problem z instalacją paczek? Użyj venv**
>
> Venv (wirtualne środowisko) jest potrzebny tylko wtedy, gdy `pip install` zgłasza błąd (np. `externally-managed-environment` na macOS lub Linuxie). To osobny folder z własnym Pythonem i paczkami tylko dla tego projektu, który nie dotyka systemowego Pythona. Utwórz go raz, aktywuj w każdym nowym oknie terminala (na początku wiersza pojawi się `(.venv)`), a potem instaluj i uruchamiaj jak zwykle:
>
> ```
> # macOS / Linux:  python3 -m venv .venv  &&  source .venv/bin/activate
> # Windows:        python -m venv .venv  &&  .venv\Scripts\Activate.ps1
> pip install -r requirements.txt
> ```

4 Jak działa program

Cała logika jest w `monitor.py`. Przeczytaj go, zanim zaczniesz pisać testy. Ma trzy funkcje:

- `pobierz_temperature(nazwa)` – pyta serwer o temperaturę basenu,
- `decyzja(temperatura, norma_min, norma_max)` – zwraca `OK`, `PODGRZEJ`, `SCHLODZ` albo `ALARM`,
- `sprawdz_basen(nazwa)` – łączy obie i zwraca słownik, np. `{'basen': 'duzy', 'temperatura_c': 27, 'akcja': 'OK'}`.

Normy i progi (tolerancja 3 °C: poza normą, ale w tolerancji, to podgrzewanie lub schładzanie; dalej to ALARM):

Basen (nazwa w kodzie)	Norma [°C]	Za zimno / za ciepło	ALARM
`duzy`	26 – 28	PODGRZEJ / SCHLODZ	poniżej 23 lub powyżej 31
`jacuzzi`	36 – 39	PODGRZEJ / SCHLODZ	poniżej 33 lub powyżej 42
`zimny`	10 – 15	PODGRZEJ / SCHLODZ	poniżej 7 lub powyżej 18
`dzieciecy`	29 – 31	PODGRZEJ / SCHLODZ	poniżej 26 lub powyżej 34

5 Jak wygląda test

```
from monitor import sprawdz_basen


def test_duzy_basen_jest_w_normie():
    wynik = sprawdz_basen("duzy")
    assert wynik["akcja"] == "OK"
```

Nazwa funkcji musi zaczynać się od `test_` i mówić, co sprawdzamy. `assert` porównuje wynik z oczekiwaniem: gdy warunek jest fałszywy, test jest czerwony. Funkcję `decyzja()` testujesz bez serwera, np. `decyzja(27, 26, 28) == "OK"`.

6 Zadanie

W `tests/test_basen.py` jest komentarz `TODO`. Dopisz testy dla:

1. **jacuzzi** – w normie, oczekiwana akcja `OK`,
2. **zimny** – ten basen jest za ciepły (norma 10–15 °C), więc ma zostać schłodzony: akcja `SCHLODZ`.

Wskazówka: skopiuj istniejący test i zmień nazwę basenu, oczekiwaną akcję i nazwę testu.

> **Kryteria „zrobione”**
>
> - W pliku są testy dla wszystkich czterech basenów, każdy z czytelną nazwą.
> - Polecenie pytest kończy się bez błędów.
> - Potrafisz wytłumaczyć, co dokładnie sprawdza każdy Twój test.

Gdy masz zielone testy, zgłoś się do prowadzącego.

7 Zadanie dodatkowe

Dopisz testy funkcji `decyzja()` dla wartości brzegowych: dokładnie na granicy normy (26 i 28), na granicy tolerancji (23 i 31) i tuż za nią. Sprawdź, co zwraca funkcja i czy zgadza się to z Twoimi oczekiwaniami. Zastanów się też, czy któryś z Twoich testów mógłby przejść, gdy program jest zepsuty.

8 Praca z AI

Możesz użyć asystenta AI, ale celem jest, żebyś nauczył się oceniać jego wyniki:

- przeczytaj każdy wygenerowany test i powiedz własnymi słowami, co sprawdza,
- zapytaj siebie, skąd test „wie”, jaki wynik jest poprawny: z wymagań czy tylko z tego, co zwraca kod,
- sprawdź, czy AI nie wymyśliło funkcji lub opcji, których nie ma w projekcie. To Ty odpowiadasz za oddane testy.

9 Gdy coś nie działa

Problem	Co zrobić
pip: command not found / pytest: command not found	Spróbuj python3 -m pip install -r requirements.txt oraz python3 -m pytest. Na Windows spróbuj py zamiast python.
error: externally-managed-environment	Pip blokuje instalację do systemowego Pythona. Użyj venv (ramka powyżej).
ModuleNotFoundError: No module named 'monitor'	Uruchom pytest z głównego folderu projektu (tam, gdzie jest plik pytest.ini), a nie z folderu tests.
Timeout lub ConnectionError	Sprawdź internet i Wi-Fi. Jeśli nadal nie działa, zgłoś się do prowadzącego.

Linki: pytest · jak działa assert · venv · Codespaces